

Praktikum 9 - Matakuliah Pilihan 1 (Web)

Program Studi: Teknik Informatika

Lakukan praktikum dibawah ini, dan buat screenshot untuk pembuktian mengerjakan setiap poin dengan mengisi tabel dibawah, kemudian tunjukan hasil akhir dari men-share repository github yang telah dibuat.

A. Membuat JSON Web Token (Dynamic Bearer Token)

1. Lanjutkan Project Praktikum 8-9, dengan menggunakan file yang sama (copy)
2. Install library JWT
npm install jsonwebtoken bcryptjs
3. Tambahkan file [auth.controller.js](#), [auth.middleware.js](#), dan [auth.routes.js](#)
4. Buat file .env disamping [server.js](#) (root folder)
Isi file .env dengan variable sebagai berikut:
JWT_SECRET="KUNCI-RAHASIA"
JWT_EXPIRE=1d
5. Tambahkan script berikut di server.js

```
require('dotenv').config();
```
6. Revisi model sebelumnya pada [user.model.js](#) dengan menambahkan fungsi baru seperti berikut, tambahkan findByEmail

```
delete: (id, callback) => {
  db.query('DELETE FROM users WHERE id = ?', [id], callback);
},

// Get user by Email (untuk login)
findByEmail: (email, callback) => {
  db.query('SELECT * FROM users WHERE email = ?', [email], callback);
},

};
```

7. Masukkan script berikut pada [auth.controller.js](#) yang telah dibuat

```
JS auth.controller.js U X
controllers > JS auth.controller.js > login > login > User.findByEmail() callback
1  const User = require('../models/user.model');
2  const bcrypt = require('bcryptjs');
3  const jwt = require('jsonwebtoken');
4
5  exports.login = (req, res) => {
6    const { email, password } = req.body;
7
8    User.findByEmail(email, (err, results) => {
9      if (err) return res.status(500).json({ message: err.message });
10     if (results.length === 0) return res.status(404).json({ message: "User not found" });
11
12     const user = results[0];
13
14     const match = bcrypt.compareSync(password, user.password);
15     if (!match) return res.status(400).json({ message: "Wrong password" });
16
17     const token = jwt.sign(
18       { id: user.id, email: user.email },
19       process.env.JWT_SECRET,
20       { expiresIn: "7d" }
21     );
22
23     res.json({
24       message: "Login success",
25       token,
26       user: { id: user.id, name: user.name, email: user.email }
27     });
28   });
29 };
```

8. Ubah [auth.middleware.js](#) yang sebelumnya menggunakan token biasa, menjadi json web token seperti gambar dibawah ini

```
h.controller.js U    JS user.model.js M    JS auth.middlewares.js M X
middlewares > JS auth.middlewares.js > ...
const jwt = require("jsonwebtoken");
const User = require("../models/user.model");

module.exports = (req, res, next) => {
  const header = req.headers.authorization;

  if (!header || !header.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Unauthorized" });
  }

  const token = header.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // Optional: cek user masih ada
    User.getById(decoded.id, (err, results) => {
      if (err) return res.status(500).json({ message: err.message });
      if (results.length === 0) {
        return res.status(401).json({ message: "Invalid token user" });
      }

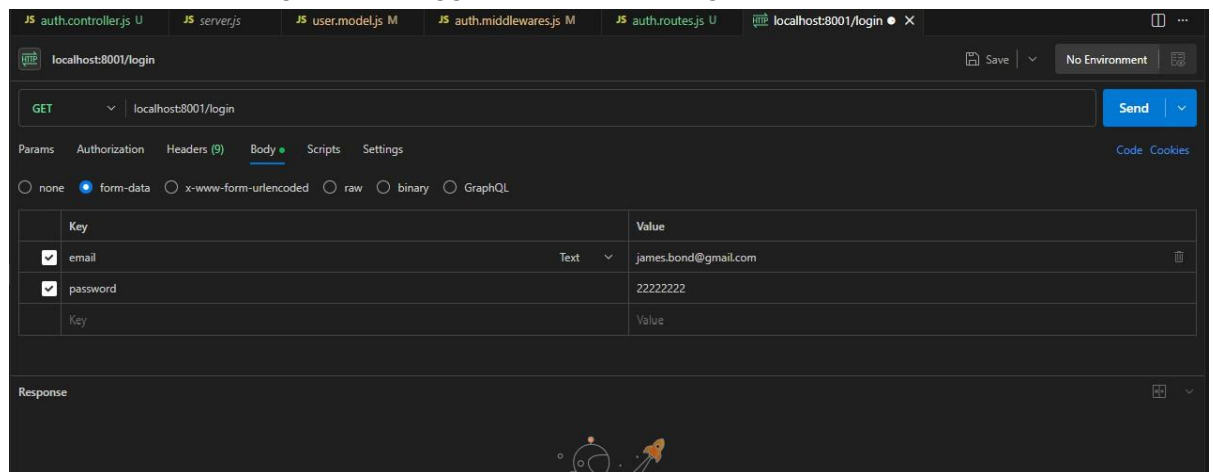
      req.user = results[0];
      next();
    });
  } catch (err) {
    return res.status(401).json({ message: "Invalid token" });
  }
};
```

9. Tambahkan Routes untuk mengakses login pada auth.routes.js

```
JS auth.controller.js U JS user.model.js M JS auth.middlewares.js M JS auth.routes.js U X
routes > JS auth.routes.js > ...
1  const express = require("express");
2  const router = express.Router();
3  const authController = require("../controllers/auth.controller");
4
5  router.post("/login", authController.login);
6
7  module.exports = router;
```

B. Gunakan POSTMAN dapatkan Token BEARER

1. Install postman di visual code, dan lakukan login berdasarkan email dan password yang terdaftar di database
2. Dapatkan bearer dengan memanggil API endpoints /login



3. Catat bearer yang di dapatkan, lalu gunakan bearer tersebut untuk memanggil endpoints lainya yang pada praktikum 9 telah di proteksi.

F. Github + Visual Code

1. Buat proyek di Github dengan nama **Latihan9**

git init

git add .

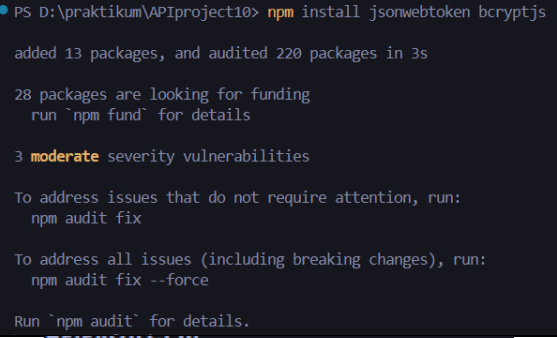
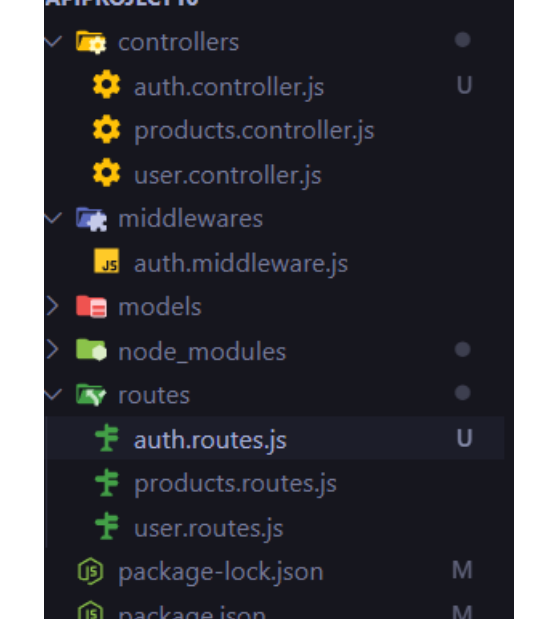
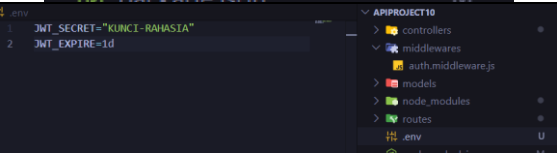
git commit -m "first commit"

git branch -M main

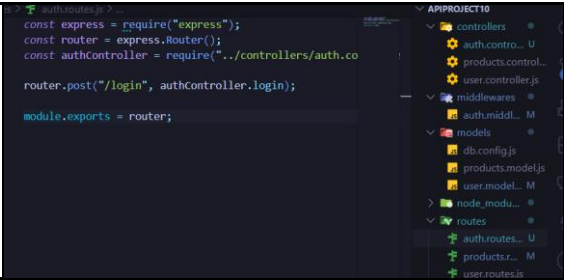
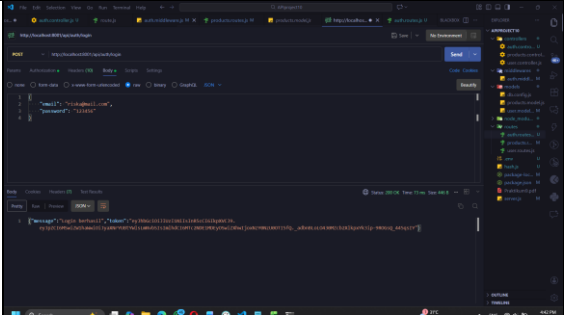
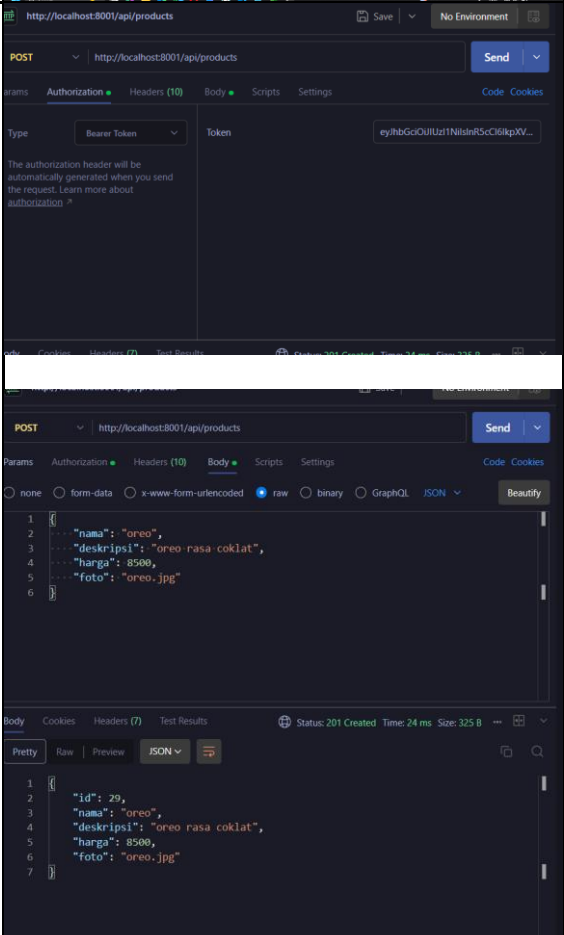
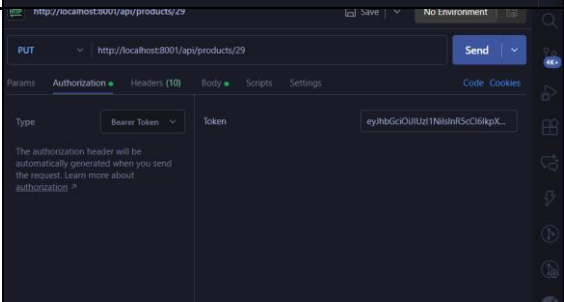
git remote add origin https://github.com/agunghakase/Latihan9.git

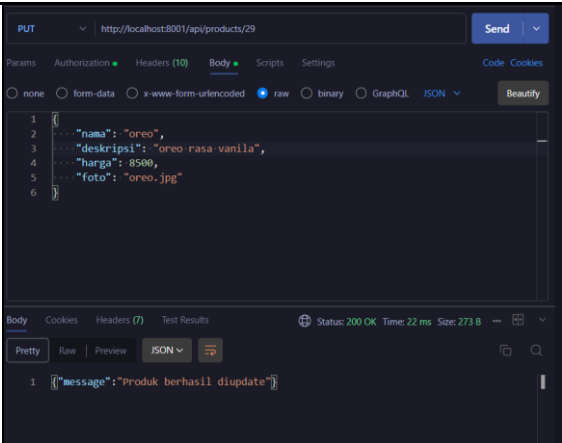
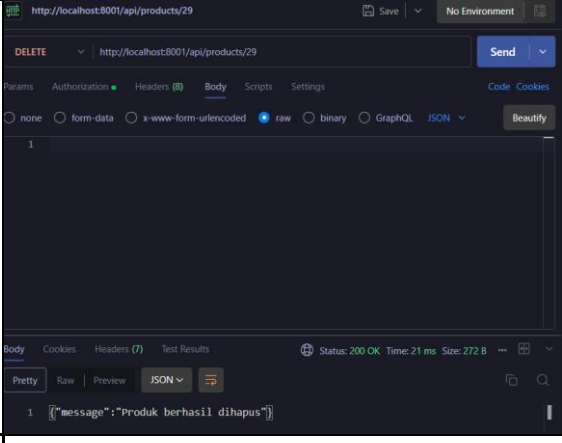
git push -u origin main

Hasil Pengerjaan

No.	Instruksi	Screenshot	Kendala/Saran
A.	Instalasi dan Konfigurasi		
1.	Intall library JWT		-
2.	Membuat file auth.controller.js, auth.middleware.js, dan auth.routes.js		-
3.	Buat file .env		-

5.	Edit server.js	<pre> server.js > ... You, 14 minutes ago • 1 author (you) require('dotenv').config(); const express = require('express'); const app = express(); const port = 8001; app.use(express.json()); app.use(express.urlencoded({ extended: true })); // Routes User const userRoutes = require('./routes/user.routes'); app.use('/api/users', userRoutes); // Routes Produk const productRoutes = require('./routes/products.routes'); app.use('/api/products', productRoutes); // Routes Login const authRoutes = require('./routes/auth.routes'); app.use('/api/auth', authRoutes); app.get('/', (req, res) => { res.send('Hello, world!'); }); app.listen(port, () => { console.log('Server is running on http://localhost'); }); </pre>	
6.	Edit file user.model.js	<pre> 1 const db = require('./db.config'); 2 3 const User = { 4 getAll: (callback) => { 5 db.query('SELECT * FROM users', callback); 6 }, 7 8 getById: (id, callback) => { 9 db.query('SELECT * FROM users WHERE id = ?', [10], callback); 11 }, 12 13 create: (data, callback) => { 14 db.query('INSERT INTO users SET ?', data, callback); 15 }, 16 17 update: (id, data, callback) => { 18 db.query('UPDATE users SET ? WHERE id = ?', [data, id], callback); 19 }, 20 21 delete: (id, callback) => { 22 db.query('DELETE FROM users WHERE id = ?', [id], callback); 23 }, 24 25 // Get user by Email (untuk login) 26 findByEmail: (email, callback) => { 27 db.query('SELECT * FROM users WHERE email = ?', [email], callback); 28 }, 29 }; 30 31 module.exports = User; </pre>	
7.	Buat file auth.controller.js	<pre> auth.controller.js > ... const jwt = require('jsonwebtoken'); const bcrypt = require('bcryptjs'); const User = require('./models/user.model'); exports.login = (req, res) => { const { email, password } = req.body; User.findByEmail(email, (err, results) => { if (err) return res.status(500).json({ message: 'Error' }); if (!results results.length === 0) { return res.status(404).json({ message: 'User not found' }); } const user = results[0]; bcrypt.compare(password, user.password, (err, isMatch) => { if (err) return res.status(500).json({ message: 'Error' }); if (!isMatch) { return res.status(401).json({ message: 'Invalid password' }); } const token = jwt.sign({ id: user.id, email: user.email }, process.env.JWT_SECRET, { expiresIn: '7d' }); res.json({ message: 'Login berhasil', token: token }); }); }); }; </pre>	
8.	Edit file auth.middleware.js	<pre> auth.middleware.js > ... const jwt = require('jsonwebtoken'); const User = require('./models/user.model'); module.exports = (req, res, next) => { const header = req.headers.authorization; if (!header !header.startsWith('Bearer ')) { return res.status(401).json({ message: 'Unauthorized' }); } const token = header.split(' ')[1]; try { const decoded = jwt.verify(token, process.env.JWT_SECRET); // Optional: cek user masih ada User.getById(decoded.id, (err, results) => { if (err) return res.status(500).json({ message: 'Error' }); if (!results results.length === 0) { return res.status(401).json({ message: 'User not found' }); } }); req.user = results[0]; next(); } catch (err) { return res.status(401).json({ message: 'Invalid token' }); } }; </pre>	

9.	Buat route auth.routes.js		-
10.	Test login untuk dapatkan token bearer		-
11.	Tes untuk tambah product		-
12.	Tes untuk edit produk		-

			
13.	Tes untuk hapus produk		
B.	Github dan Viscode		
1.	Upload github	